

# Software Maintenance

- Software Maintenance is the process of modifying a software product after it has been delivered to the customer.
- The main purpose of software maintenance is to modify and update software application after delivery to correct faults and to improve performance.

# Need for Maintenance –

- Software Maintenance must be performed in order to:
- Correct faults.
- Improve the design.
- Implement enhancements.
- Interface with other systems.
- Accommodate programs so that different hardware, software, system features, and telecommunication facilities can be used.
- Migrate legacy software.
- Retire software.

# Categories of Software Maintenance –

Maintenance can be divided into the following:

**1. Corrective maintenance:** Corrective maintenance of a software product may be essential either to rectify some bugs observed while the system is in use, or to enhance the performance of the system.

**2. Adaptive maintenance:** This includes modifications and updations when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.

- **3. Perfective maintenance:** A software product needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer demands.
- **4. Preventive maintenance:** This type of maintenance includes modifications and updates to prevent future problems of the software. It goals to attend problems, which are not significant at this moment but may cause serious issues in future.

# Important points to be noted:

- During maintenance it is necessary to thoroughly understand someone else's work and then carry out the required modifications and extensions.
- Another problem associated with maintenance work is that the majority of software products needing maintenance are legacy products.

# Discussion Items

- Introduction: What is Software Maintenance?
- SM – Root Problem
- SM – Evolution
- Problems of SM
- Organizational Aspect of SM
- IEEE Draft Standard for SM
- Technical Aspects of SM
- Legacy Systems
- Reverse Engineering
- Conclusion

# What is Software Maintenance?

- IEEE91 Definition:
  - the process of modifying the software system or component after delivery to correct faults, improve performance or other attributes, or adapt to a change in the environment.
- SM is concerned with modifying software once it is delivered to a customer
- Major economic importance: 40 – 90% of the total lifecycle costs

# What is Software Maintenance?

- Four categories:
  1. **Perfective maintenance:** changes required as a result of user requests (a.k.a. *evolutionary* maintenance)
  2. **Adaptive maintenance:** changes needed as a consequence of operated system, hardware, or DBMS changes
  3. **Corrective maintenance:** the identification and removal of faults in the software
  4. **Preventative maintenance:** changes made to software to make it more maintainable
- The majority of SM is concerned with evolution deriving from user requested changes.

# What is Software Maintenance?

- What is meant by SM for organization:
  - a major economic importance
  - a substantial applications backlog
- Starting for late 1960s, SM started to be recognized as a significant activity

# Software Maintenance: The Root Problem

- The root problem is **complexity**.
- Sometimes complexity arises because a system is migrated from hardware to software in order to gain the additional functionality found in software.
- The combination of scale and application complexity mean that it is infeasible for one person alone to understand the complete software system.

# Software Maintenance: Evolution

- The important requirement of software maintenance for the client is that changes are accomplished *quickly & cost effectively*.
  - Reliability should not degrade.
  - Maintainability should not degrade.
- Maintenance that becomes increasingly more expensive and difficult becomes known as a legacy system.
  - The legacy system may still be of essential importance to the organization.

# Problems of SM

1. Alignment with organizational objectives.
  - SM is resources consuming and it has no clear quantifiable benefit for the organization.
2. Process issues
  - SM requires a number of additional activities not found in initial development. Impact analysis and regression tests on the software changes are crucial issues.
3. Technical issues
  - How to construct software that it is easy to comprehend is a major issue and the technology to do this is still not available.

# Problems of SM

- Domino Effect (Ripple Effect)
  - When a change is made to the code, there may be substantial consequential changes, not only in the code itself, but within documentation, design, and test suites.
- SM usually has a lower status compared with software development.
- Management has trouble assessing a software product to determine how easy it is to change:
  - This leaves little incentive for initial development projects to construct software that is easy to evolve.

# Organizational Aspect of SM

- SM is much closer to a service and be related to quality
  - As opposed to initial software development which is product-oriented.
- SM requires financial investment
  - SM is a prime candidate for funding reduction and even elimination.
  - SM needs to be expressed in terms of ROI.
- The trend for SM to be outsourced
- Work has been undertaken in applying predictive cost modeling to software maintenance
  - Based on the COCOMO techniques.

# IEEE Draft Standard for SM

## [IEEE94]

- Represents many elements of good practice in SM:
  - Accepting a stream of change requests & error reports
  - Implementing the changes
  - Testing
  - Forming new software releases
- IEEE draft model comprises four key stages:
  - Help desk: the problem is received, a preliminary analysis undertaken, and if the problem is sensible, it is accepted.
  - Analysis: a managerial and technical analysis of the problem is undertaken to investigate and cost alternative solutions.
  - Implementation: the chosen solution is implemented and tested.
  - Release: the change (along with others) is released to the customer.

# IEEE Draft Standard for SM [IEEE94]

- Analysis phase
  - Feasibility analysis
    - Assesses the impact of the modification, investigates alternative solutions, assesses short and long term costs, and computes the benefit value of making the change.
  - Detailed analysis
    - Determines firm requirements of the modification, identifies the software involved, and requires a test strategy and an implementation plan to be produced.
- The standard requires that:
  - All infected components shall be identified and brought in to the scope of the change; Unit test, integration test, user-oriented functional acceptance tests and regression test strategy be provided.

# Technical Aspects of SM

- **Impact Analysis** – helps to determine the cost of making a change
  1. Translate the problem into software terms to decide if it is viable or be rejected
  2. Determine the origin of the change and suggest solutions
  3. All solutions be investigated to determine they are applied to all software components.
  4. Make a decision on the best implementation route or to make no change.

# Technical Aspects of SM

- The Ripple Effect problem:
  - Ripple Effect propagation is a phenomenon by which changes made to a software component along the software life cycle have a tendency to be felt in other components. (Yau)
  - Ripple effects cannot determine statically, and dynamic analysis must be used.
- Impact Analysis is needed:
  - To ensure that the change has been correctly and consistently bounded; to identify all objects impacted by changes in the primary sector.

# Technical Aspects of SM

## ■ Traceability

- A degree to which a relationship can be established between two or more products of the development process, especially products having a predecessor successor or master subordinate relationship to one another. (IEEE91)
- Traceability provides semantic links for impact analysis.
- Some types of traceability links are very hard to determine.

# Legacy Systems

- Legacy problems – a legacy system is typically:
  - very old and large
  - has been heavily modified
  - based on the old technology
  - documentation not be available
  - none of the original members of the software development team may still be around
  - often support very large quantities of live data
  - the software is often at the core of the business and replacing it would be a **huge** expense.
- Dealing with legacy system is very hard and there are some solutions.

# Legacy Systems

- Solutions for the legacy system:
  - Carry on as now, possibly subcontracting the maintenance
  - Replace software with a package
  - Re-implement from scratch
  - Discard software and discontinue
  - Freeze maintenance and phase in new system
  - Encapsulate the old system and call as a sever to the new
  - **Reverse Engineer** the legacy system and develop a new software suite. → the most fruitful solution!

# Legacy Systems – Reverse Engineering

- Definition: Reversing Engineering
  - The process of analyzing a subject system to identify the system's components and their inter-relationships, and to create representations of the system in another form or at higher levels of abstraction.( by Chikofsky & Cross)
- Two types of Reverse Engineering:
  1. Redocumentation: the creation or revision of a semantically equivalent representation within the same relative abstraction layer;
  2. Design Recovery: involves identifying meaningful higher level abstractions beyond those obtained directly by examining the system itself.
- The main motivation is to provide help in program comprehension

# Legacy Systems – Reverse Engineering

- Two approaches:
  - Slicing:
    - A static analysis technique in which only those source code statements which can affect a nominated variable are displayed.
  - Hypertext form of documentation:
    - The maintainer attaches and manage notes to the source code for the “hot spots” (Foster and Munro)

# Legacy Systems – Reverse Engineering

- Reasons to use Reverse Engineering:
  - 26 decision criteria in three categories for reverse engineering: (by Bennett)
    - Management criteria
    - Quality criteria
    - Technical criteria
- Two Important Points:
  - Many legacy systems represent years of accumulated experience, and this experience may now no longer be represented anywhere else.
  - It is not obvious that a legacy system does actually have a high level, coherent representation.